DataShield Developer Guide

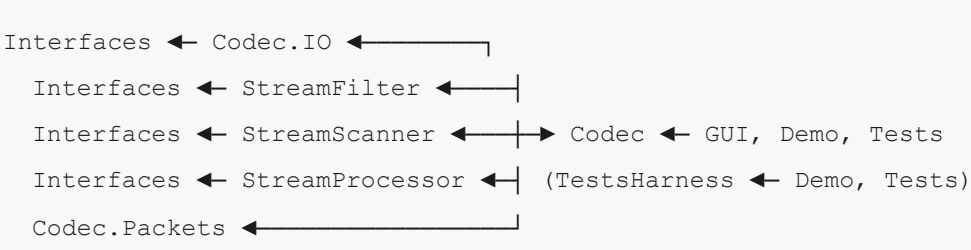
Solution layout, the logic of every module, the streaming pipeline model, tests, and the build. The algorithms (accumulation, rebinding, assembly) are covered in depth in `AlgorithmGuide.en.md`; this guide is about architecture and modules.

1. Solution Overview

`DataShield.slnx` contains 19 projects. The decoding pipeline is assembled from small assemblies, each solving one task:

Project	Purpose and logic
`DataShield.Interfaces`	Pipeline contract: `IDataSource`, `IDataProcessor`, `IDataWriter`, the `DataProcessorBase` base, `DataReadyHandler`, `TakeBufferDelegate` delegates
`DataShield.Codec.Packets`	Wire format: `PacketFormat` constants (75 B/100 chars, H/D layouts) and `HeaderContent` serialization
`DataShield.Codec.IO`	Sources: `FileSource`, `StreamSource`, `ByteArraySource` (inherit `BufferedSourceBase`); writers: `FileDataWriter`, `StreamDataWriter`, `ByteListWriter`, `PreallocatedBufferWriter` (inherit `WriterBase`)
`DataShield.Codec.StreamFilter`	`ByteRangeFilter` — byte filtering with a `bool[256]` map built from `ByteRange` sets; factory `CreateBase64()`
`DataShield.Codec.StreamScanner`	`SlidingWindowScanner` — sliding-window scanning with a `WindowHandler` delegate; whole-stream retention; deferred rescan queue
`DataShield.Codec.StreamProcessor`	Reception core: the `StreamProcessor` accumulator, the `ReceptionSlot` slot, `Versions/*` combinatorics, the `RsCodecAdapter`, `Localization`, `Progress`
`DataShield.Codec`	Facade: `FileEncoder`, `FileDecoder`, `EncodeStats` statistics, output via `Packets/PacketIO`, `OutputFormat(Config)`
`DataShield.GUI`	Avalonia UI (MVVM): `MainViewModel`, `MainWindow.axaml`, `SectorMapControl`, localization `UiStrings` / `LanguageManager`, `AppSettings`
`DataShield.Demo`	Continuous randomized stability/performance bench (PASS/WARN/FAIL)
`DataShield.Tests`	Codec integration tests (162)
`DataShield.TestsHarness`	Damage sources for tests and the bench: `DamageEngine`, `BinaryDamage`, `PacketDamage`, `LineDamage`, `DamageBits`, `PacketProbe`, `RandomInput`
`RsRaid16Demo`	Demonstration and tests of the GF(2^16)/Reed–Solomon field
`Sha256CompactDemo`	Demonstration of the compact SHA-256 implementation
`DataShield.*.Tests` × 6	Unit tests, one assembly per pipeline module (see section 8)

The **`refs-src/`** directory holds unmodified reference sources `RsRaid16.cs`, `GF16.cs`, `Sha256Compact.cs`; they are compiled into the projects that need RS/SHA. Editing the references is not allowed. Dependency graph (simplified):



Namespace caveat: the types of the `DataShield.Codec.StreamProcessor` assembly live in namespaces `DataShield.Codec` / `DataShield.Codec.Versions` — the facade namespaces were preserved when the monolith was decomposed. Namespace ≠ assembly name.

2. The Pipeline Contract (`DataShield.Interfaces`)

All data movement is built on three roles:

- **`IDataSource`** — a buffered, event-driven source. `Start()` begins reading into the `BufferSize` buffer; a filled buffer is announced via the `DataReady` event, and reading pauses until the client drains the buffer (natural backpressure — the source never runs ahead). On EOF the remainder is emitted and the source stops by itself. `Stop()` is an external stop; the buffer remainder is still emitted.
- **`IDataProcessor : IDataSource`** — a "black box": it has an input (`Attach/Detach(IDataSource)`) and its own output; results are published through its own `DataReady`, enabling chains. `Complete()` — the end-of-input signal: the output-buffer remainder is flushed and the processor stops.
- **`IDataWriter`** — a terminal sink: `Write(ReadOnlySpan<byte>)`, attaching to a source via `Attach`.

`DataProcessorBase` — the shared processor implementation: upstream attachment with cascading `Start/Stop/Complete` (the completion signal travels down the whole chain), an output buffer of `BufferSize`, and two emission modes:

- `Emit(bytes)` — a byte stream, buffering allowed (portions may be split/merged freely);
- `EmitPacket(bytes)` — **indivisible** portions (packets): they must never be split and must be emitted whole.

Thread safety is provided by `SyncRoot`; the pipeline as a whole is single-threaded with respect to data (calls propagate along the event chain), so synchronization is only needed where external code reads state concurrently (e.g. `StreamProcessor`).

3. `Codec.Packets` — the Wire Format

`PacketFormat` is constants only (sizes, field offsets H1–H5/D1–D3, `Base64Size = 100`, `MaxFileSizeField = 16,777,215`). `HeaderContent` is a readonly record struct with 51-byte serialization (`ToBytes/WriteTo/ReadFrom`, UInt24LE for the size, space-padded name) and computed `DataVolumeCount` / `TotalVolumeCount`. `PacketHasher` computes the truncated SHA-256 integrity hashes (H5 = 24 bytes over the header content, D3 = 9 bytes over H5 ‖ D1 ‖ D2). `FileNameCodec` packs a name into the 14-byte H1 field. There is no decision logic here — the module depends on nothing but the BCL.

4. `Codec.IO` — Sources and Writers

Sources (`BufferedSourceBase`) implement the `IDataSource` event model over concrete stores: array (`ByteArraySource`), stream (`StreamSource`), file (`FileSource`). Writers (`WriterBase`) write to a file/stream/list/preallocated buffer. Used by the `FileDecoder` facade (input: `ByteArraySource`, `Stre`

amSource) and by output consumers.

5. Codec.StreamFilter and Codec.StreamScanner

ByteRangeFilter — an IDataProcessor with a bool[256] allowed-byte map built from ByteRange sets. It passes bytes inside the ranges and drops the rest. CreateBase64() is a ready filter of the Base64 alphabet for text mode (line breaks, spaces, and junk vanish before decoding). In binary mode the filter is not part of the chain.

SlidingWindowScanner — an IDataProcessor that processes input with a fixed-length window (100 bytes for Base64, 75 for binary) and a delegate WindowHandler(window, out emitted) → int:

- the return value is the stream advance in bytes, minimum 1; the typical case: success → packet length, failure → 1. It is used by the direct pass only: a re-scan (rescan) ignores it and always shifts by 1 byte;
- emitted is a copy of the recognized packet (emitted via EmitPacket) or null.

The key feature is **whole-stream retention**: the scanner never discards processed bytes while it lives. This enables RequestRescan(handler) — an exhaustive repeated pass over retained data with a different window: the window is checked at every position and the delegate's advance is ignored (used for the targeted rebinding of sectors that arrived before their header; completeness is mandatory because the direct pass's post-success jump can skip the start of an overlapping valid packet). The deferred-rescan queue is bounded by the last direct-emission boundary: a re-scan never goes past it, so the same data cannot be confirmed twice. The ConsumedAdvanced event reports stream-consumption progress.

6. Codec.StreamProcessor — the Reception Core

Contents: StreamProcessor (accumulator), ReceptionSlot (file slot), Versions/ (SectorCombinationMath, SectorVersionSearchOptions, ChoicePoint, SectorVariant, SectorVersionInfo), RsCodecAdapter (RS over GF(2^16), references from refs-src), Localization (CodecStrings), Progress (CodecProgress, ScaledProgress, ProgressThrottle).

Logic:

- **StreamProcessor : DataProcessorBase** stitches arbitrary input chunks into whole 75-byte packets (a _pending buffer across chunk boundaries), classifies each one (autonomous hash → header; H5-seeded hash → a sector of a specific slot; a sector may match several slots), and maintains the ReceptionSlot list. A new header → a new slot plus the HeaderAccepted(header, headerHash) event (raised outside the lock). Snapshot properties (Slots, FileCount, aggregate counters) are read under the lock concurrently with reception. Recognizes(packet) is a side-effect-free recognition predicate for the scanner's window delegate.
- **ReceptionSlot** — a SortedDictionary<sector number, List<SectorVariant>>; payload versions sorted by descending confirmation count; AddSector either increments the matching version's counter (with "bubbling up") or appends a new version at the end. It provides metrics

(coverage, validity/collision maps) and the `TryAssemble` assembly (direct → RS → combination search/rotation; details in `AlgorithmGuide.en.md`, section 9, and the `AssemblyGuide.Academic.en.md` / `AssemblyGuide.Engineer.en.md` monographs).

- **`Versions`** — pure combinatorics: the `AdvanceIndexes` odometer, `CountCombinations` (saturating at long.MaxValue), `CountRotationStates` (capped LCM of cycles), `ShouldUseExhaustiveSearch`; the limits of `SectorVersionSearchOptions` (100,000 combinations, 100,000 states, 30 s).
- **`RsCodecAdapter`** — K→M encoding and erasure recovery for 64-byte volumes (32 independent GF symbols per volume); recovery condition: erased data ≤ available ECC; K+M ≤ 65,535.

7. `Codec` (Facade), `GUI`, `Demo`

`FileEncoder`: file → N data volumes → M ECC volumes → sector packets plus H header copies (first/last/evenly spaced). Progress phases 0–10–75–100, wipe of intermediate buffers. `EncodeToText` — Base64 lines; `EncodeWithStats` — plus `EncodeStats` (SHA, counters, header copies).

`FileDecoder`: assembles the pipeline `source → (Base64 filter) → scanner → accumulator`; accepts Base64 strings, raw bytes, and a `Stream` with an explicit `OutputFormat`. On `HeaderAccepted` it asks both scanners (txt and bin) to rebind retained data with the `RebindWindow` window (a sector of the late header: number range + H5-seeded hash). Assembly is `TryAssemble(HeaderContent)`: find the slot by byte-comparing the serialized header and call `ReceptionSlot.TryAssemble`. Reception state is exposed via `Slots` / `FileCount` for UI indication.

`DataShield.GUI` — an Avalonia application (framework-free MVVM): `MainViewModel` + `RelayCommand`, the `SectorMapControl` sector map, bilingual UI `UiStrings` / `LanguageManager` / `UiLanguage`, converters, `AppSettings`, `WorkMode` modes. All heavy codec operations go through the `DataShield.Codec` facade.

`DataShield.Demo` — an endless bench: a random file (1 B–256 KB, log-uniform), ECC 1–200%, a random `DamageBits` damage mask, a PASS/WARN/FAIL expectation matrix, a ring table of iterations with speeds. WARN is a deliberate refusal (over-budget damage/forgery), FAIL is a codec defect. The main regression tool; legend details are printed by the bench itself.

8. Tests

Assembly	Coverage	Tests
`DataShield.Interfaces.Tests`	pipeline contract, `DataProcessorBase`	6
`DataShield.Codec.Packets.Tests`	packet format, header serialization	52
`DataShield.Codec.IO.Tests`	sources and writers	16
`DataShield.Codec.StreamFilter.Tests`	range filters, Base64 filter	8
`DataShield.Codec.StreamScanner.Tests`	sliding window, retention, exhaustive rescan	11
`DataShield.Codec.StreamProcessor.Tests`	accumulator, slots, versions, combinatorics, RS adapter, assembly	185
`DataShield.Tests` (integration)	encoder/decoder facade, damage, multi-file streams	162
Total		**440**

9. Build and Run

```
dotnet build DataShield.slnx -c Release
  dotnet test DataShield.slnx
  dotnet run --project DataShield.Demo -c Release
  dotnet run --project DataShield.GUI -c Release
```

The .NET 10 SDK is required. The GUI also publishes self-contained following standard Avalonia practices.

10. Conventions

- Comments and XML docs are in Russian; public types are documented fully.
- Namespaces keep the historical facade names (`DataShield.Codec`, `DataShield.Codec.Versions`) regardless of the assembly.
- The `refs-src/` references are never modified or adapted.
- New reception/assembly logic is covered by tests in the matching `*.Tests` assembly; facade changes — by the `DataShield.Tests` integration suite.